



TECHNISCHE HOCHSCHULE
OSTWESTFALEN-LIPPE
UNIVERSITY OF
APPLIED SCIENCES
AND ARTS

Introduction to React

Routing & Data Fetching

9. January 2026

Organization

Schriftliche Ausarbeitung

- 10-15 Seiten
- 1 Abgabe pro Gruppe

Abgabe: **08.04.**

Final Presentation

- 20 min. Präsentation + 10 min. Fragen
- Zeitslots stehen in Ilias (*Präsentation mit schriftlicher Zusammenfassung (Prüfung)*)
 - Gruppen können Zeitslots **eigenständig tauschen**
- Alle Gruppenmitglieder müssen vorstellen

Prüfungstermine: **13.04. & 14.04.**



Vorgaben zur Finale Presentation & Schriftlichen Ausarbeitung sind **in Ilias** hochgeladen

- [2025 Prüfungsleistung/FinalPresentation.pdf](#)
- [2025 Prüfungsleistung/SchriftlicheAusarbeitung.pdf](#)

Organization

- 09.01.2026 – **Routing & Data Fetching**
- 16.01.2026 – **Context & Architecture**
- 23.01.2026 – **Beyond React**

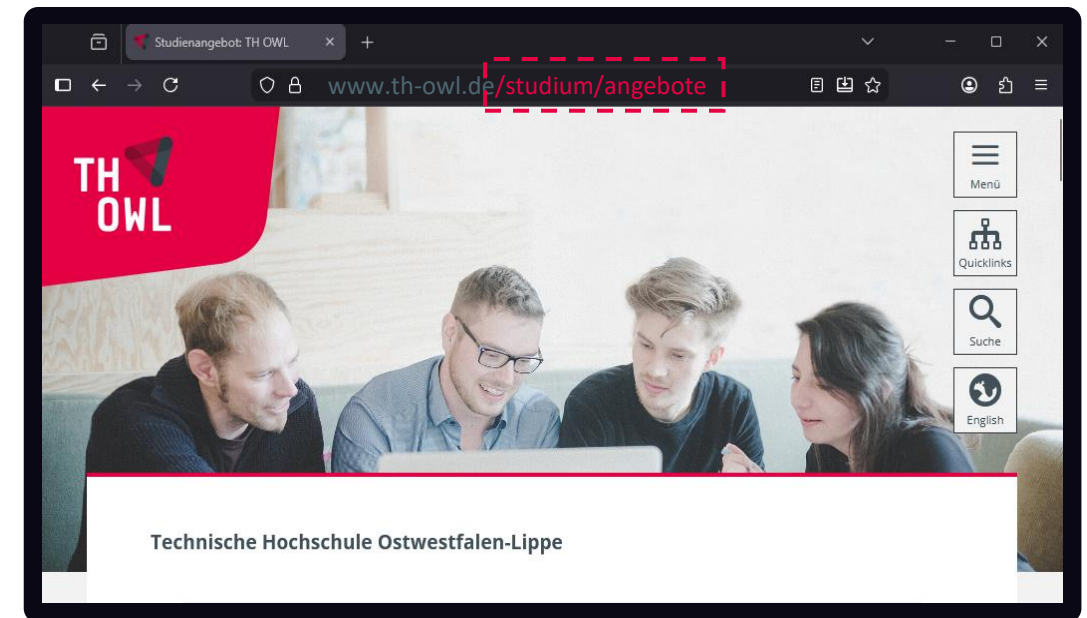


Routing

What is Routing?

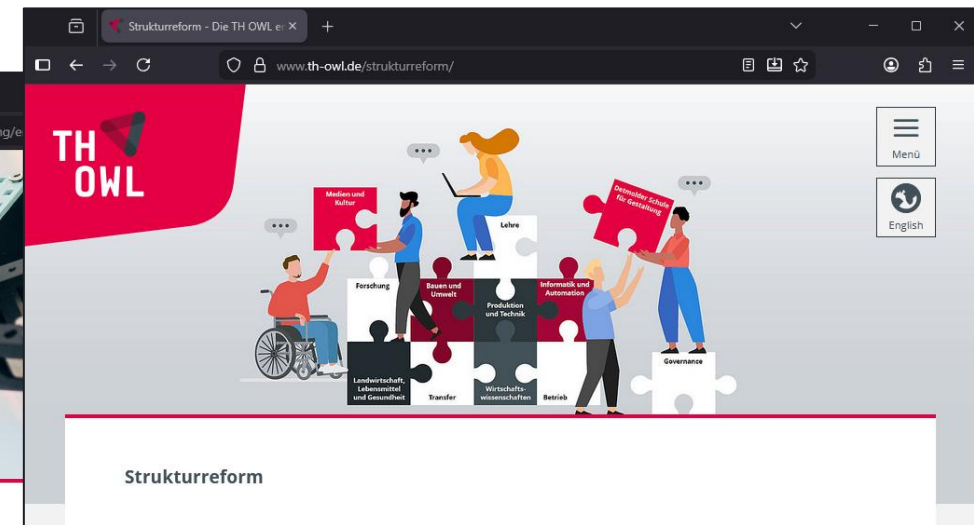
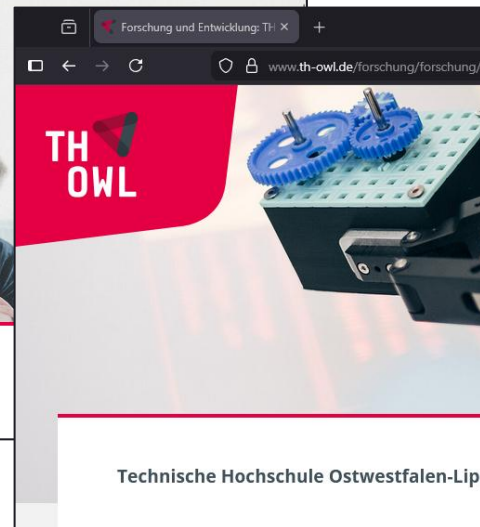
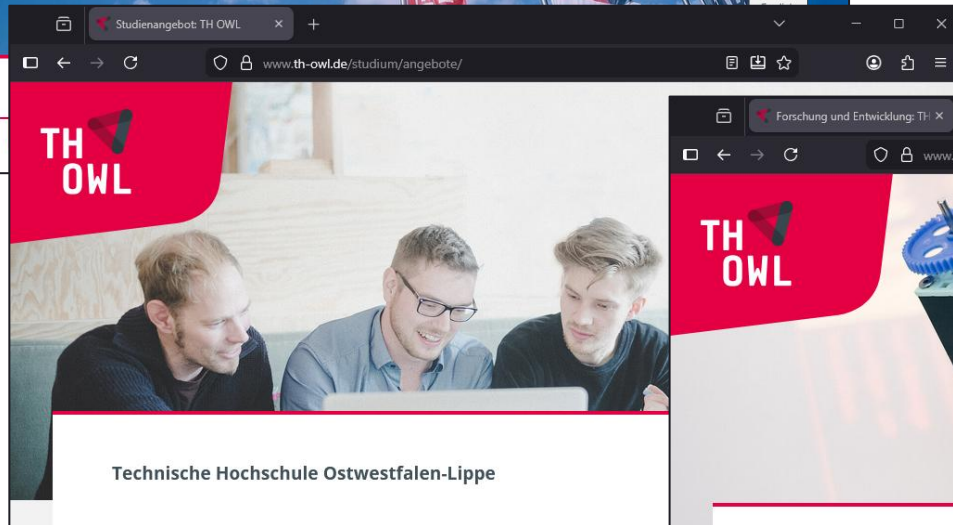
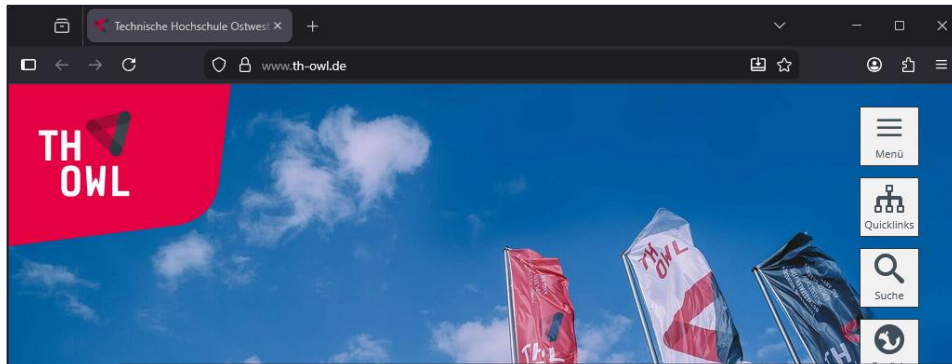
What is client-side Routing?

- Routing decides **what UI is shown** for a **given URL**
- Client-side Routing happens **in the browser**, not the server
- The page is **loaded once** (*Single-Page Application*)
- Changing the URL does not reload the page
- **Feels faster** and more like a native app



What is client-side Routing?

Navigation





Routing vs. multi-page Applications

Multi-page Applications

- **Every navigation** triggers **full page reload**
- Server returns **new HTML** for each URL
- Browser **resets JavaScript states**
- Simpler, but **slower experience**

Client-side Routing

- Page **loads once**
- URL changes handled by JavaScript
- **Only components change**
- **Application state stays**
- **Faster, smoother experience**



Why React needs a Router?

- React only renders **components**
- React **does not** know about URLs
- **Without** a router, you must **manually** handle **URLs**
 - → This is very complex and error-prone
- A Router **connects URLs** to **components**



URLs as State

- The URLs represents the current application state
- It answers: *“Where am I in the app?”*
- URLs are
 - **Bookmarkable**
 - **Shareable**
 - **Restorable** on reload



Typical Routing Use-Cases

■ Pages

- `/home`, `/about`, `/settings`, `/profile`

■ Detail Views

- `/users/42`
- `/product/21`

■ Steps

- `/checkout/cart` → `/checkout/payment` → `/checkout/confirm`

■ Authentication

- `/login`, `/register`, `/reset-password`

■ Filters & Search

- `/books?q=react&sort=rating`

React Router Basics



What is React Router?

- **React Router** is the **standard routing library** for React
- It enables client-side routing in Single-Page Applications
- Router maps URLs to components
- We will be using **Declarative Mode**

Execute in your project root:

```
npm i react-router
```

More information on <https://reactrouter.com/start/data/installation>

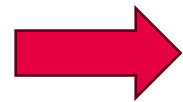
BrowserRouter vs. HashRouter

BrowserRouter

- Uses the HTML5 **History API**
- **Clean URLs**
 - `/users/42`
- *Requires server configuration (SPA)*
- **Recommended** for most real apps

HashRouter

- Uses the **URL hash (#)**
- URLs look like
 - `/#/users/42`
- *No server configuration needed*
- Works even on **static file hosts**



We will be using the BrowserRouter

Router Setup

```
1 import { BrowserRouter, Route, Routes } from "react-router"; } Import components
2
3 import Home from "./home";
4 import Users from "./users";
5
6 function App() {
7   return (
8     <BrowserRouter>
9       <Routes>
10        <Route path="/" element={<Home />} /> } <Route> defines the different
11        <Route path="/users" element={<Users />} /> url-component mappings
12      </Routes>
13    </BrowserRouter>
14  );
15 }
16
17 export default App;
```

<BrowserRouter>

Defines which router to use

**<Route> defines the different
url-component mappings**

Routes

- A `<Route>` couples a URL to a component

```
1 <Route
2   path="/users" ——— URL path
3   element={<Users />} ——— Component to render
4 />
```


Route Matching

- Routes are **matched based** on the **URL**
- The **most specific** route wins
- Only one route is rendered at a time

```
1 <Route path="/home" element={<Home />} />;  
2 <Route path="/users" element={<Users />} />;  
3 <Route path="/users/current" element={<CurrentUsers />} />;
```

1. localhost:5173/home

2. localhost:5173/users/current

3. localhost:5173/users/non-existing

No match!
→ Renders nothing

404 / Fallback

- A 404 page is shown when **no route matches** the URL
- Implemented using a wildcard path: **path="*"**
- **Prevents blank screens** on invalid URLs
- Should exist **exactly once** per app

```
1 <Routes>
2   <Route path="/" element={<Home />} />
3   <Route path="/users" element={<Users />} />
4
5   {/* 404 fallback */}
6   <Route path="*" element={<NotFound />} />
7 </Routes>;
```

Default Routes

- A default route has **no path** of its own
- Implemented using the **index** prop
- Commonly used in nested routing

```
1 <Route index element={<Home />} />;
```

Instead of

```
1 <Route path="/" element={<Home />} />;
```

Navigation

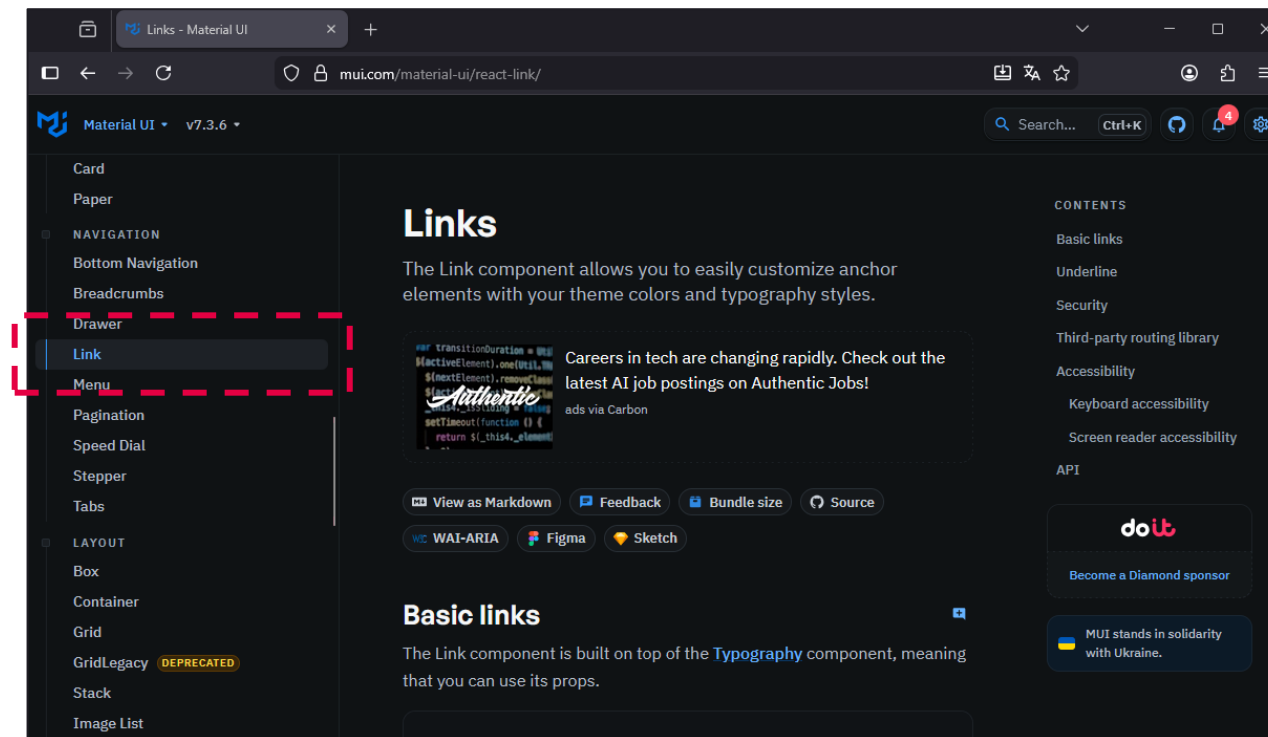
<Link> vs. <NavLink>

- **<Link>** component from **MaterialUI**
 - Triggers full **page (re)load**
 - Not aware of the current URL
 - Use for **external links**
- **<NavLink>** component from **React Router**
 - **No** page (re)load
 - Aware of the current URL
 - Use for **internal links**

```
1 import { NavLink } from "react-router";  
2  
3 <NavLink to="/">  
4   Home  
5 </NavLink>;
```

Active Link Styling

- Navigation should **show** where the **user currently is**
- **<NavLink>** provides active state automatically



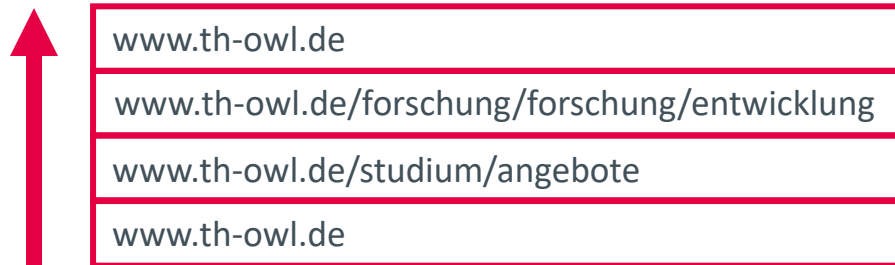
useNavigate Hook

- Sometimes navigation is triggered by **logic**, not user clicks
- **useNavigate()** hook

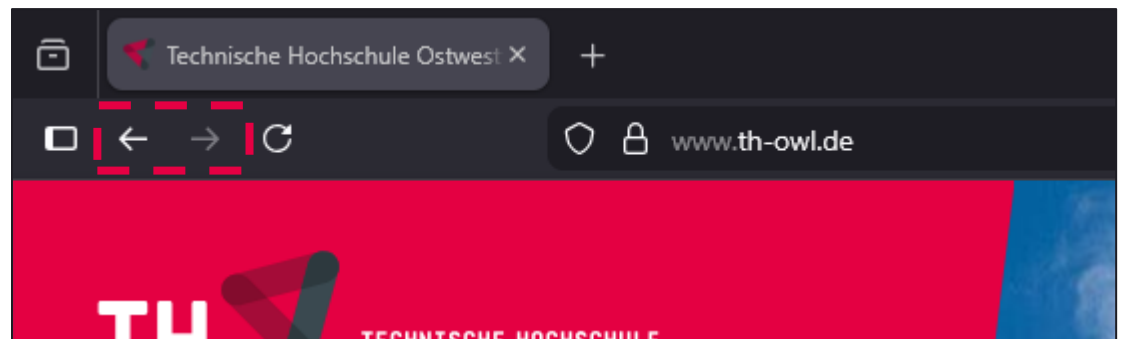
```
1 import { useNavigate } from "react-router";
2
3 const LoginButton = () => {
4   const navigate = useNavigate();
5
6   const handleLogin = () => {
7     // login logic
8     navigate("/dashboard");
9   };
10
11   return <button onClick={handleLogin}>Login</button>;
12 };
```

Navigation History

- Browser maintains a **navigation history stack**
- This includes:
 - Visited URLs
 - Back/Forward behavior
- **React Router** integrates with the **History API**

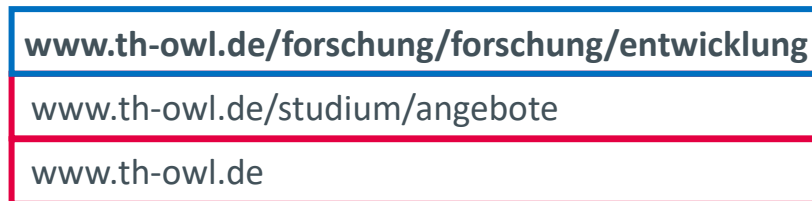


Navigation History Stack

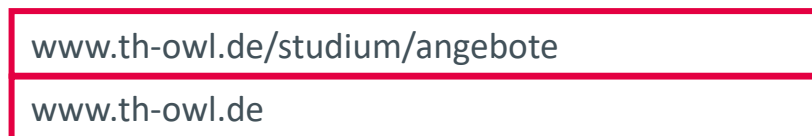


Push vs. Replace Navigation

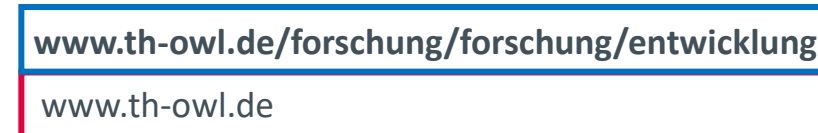
- **Push** adds a **new entry** to the history
- Users can **go back** to the previous page
- Typical for normal navigation



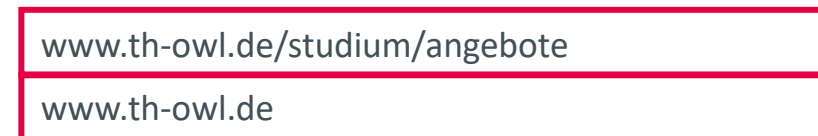
Push



- **Replace** replaces the **current history** entry
- Users **cannot** go back to the previous page
- Useful for redirects (*e.g. after login*)



Replace





Push vs. Replace Navigation

```
1 // Push (default)
2 navigate("/login");
3
4 // Replace
5 navigate("/login", { replace: true });
```

Route Parameters & URL State



Dynamic Routes

- Dynamic routes contain variable URL segments
 - e.g. `/users/1`, `/users/2`, `/users/admin`, ...
- Map **one** route definition to **many** URLs
- Defined using a **colon** (:)

```
1 <Route path="/users/:id" element={<UserDetails />} />;
```

Reading Parameters

- Route parameters are read inside the routed component
- React Router provides the **useParams** hook
- Parameters are always **strings**

```
1 <Route path="/users/:id" element={<UserDetails />} />;
```

```
1 import { useParams } from "react-router";
2
3 const UserDetails = () => {
4   const { id } = useParams();
5
6   return <div>User ID: {id}</div>;
7 };
```

Dynamic Routes Matching

```
1 <Route path="/users/:id" element={<UserDetails />} />;  
2 <Route path="/users/current" element={<CurrentUser />} />;
```

1. /users/123
2. /users/current
3. /users

No match!

Optional Parameters

- Route matches **with or without** the parameters
- Marked with an additional **question mark (?)**

```
1 <Route path="/users/:id?" element={<UserDetails />} />;
```

```
1 <Route path="/users/:id?" element={<UserDetails />} />;  
2 <Route path="/users/current" element={<CurrentUser />} />;
```

1. /users/123
2. /users/current
3. /users



Query Parameters

- **Query parameters** are the part after ? In a URL
 - e.g. `/users?q=admin`
- Multiple params are separated by **&**
 - e.g. `/users?q=admin&page=2`
- Usually used for filters, search, sorting or pagination
- They **do not** affect route matching

useSearchParams Hook

- React Router provides the **useSearchParams** hook to read and write query params

```
1 import { useSearchParams } from "react-router";
2
3 const Users = () => {
4   const [searchParams, setSearchParams] = useSearchParams();
5
6   const query = searchParams.get("q");
7   const sort = searchParams.get("sort");
8
9   return (
10     <div>
11       Search: {query}, Sort: {sort}
12     </div>
13   );
14 };
15
16 // -----
17
18 // Update query parameters
19 setSearchParams({ q: "admin", sort: "perms" });
```



URLs vs. React State

- Not all state belongs in React
- URLs are a **form of global, persistent state**
- React state is local and temporary
- Use URLs, when something is
 - **Relevant to navigation**
 - **Restorable on reload** (*Show a specific user*)
 - **Shareable** (*links, bookmarks*)

Nested & Layout Routes

What are nested Routes?

- Routes can be children of other routes
 - `/dashboard/users`
 - `/dashboard/settings`

```
1 import { Routes, Route } from "react-router";
2
3 const App = () => {
4   return (
5     <Routes>
6       <Route path="/dashboard" element={<DashboardLayout />}>
7         <Route path="users" element={<Users />} />
8         <Route path="settings" element={<Settings />} />
9       </Route>
10    </Routes>
11  );
12 };
```



Child routes **cannot** start with a slash (/)

Layout Routes

- A **layout route** is a parent route that renders **shared UI**
- Children **re-use this UI** automatically
- Only the **inner content** changes when navigating

```
1 import { Routes, Route } from "react-router";
2
3 const App = () => {
4   return (
5     <Routes>
6       <Route path="/dashboard" element={<DashboardLayout />} />
7       <Route path="users" element={<Users />} />
8       <Route path="settings" element={<Settings />} />
9     </Route>
10    </Routes>
11  );
12 };
```

<Outlet> Usage

- **<Outlet>** is a **placeholder** for **child routes**
- It tells React Router **where to render** the nested children

```
1 import { Outlet } from "react-router";
2
3 const DashboardLayout = () => {
4   return (
5     <div>
6       <header>Dashboard</header>
7
8       {/* child routes render here */}
9       <Outlet />
10    </div>
11  );
12 };
13
14 export default DashboardLayout;
```

Advanced Routing

Redirect Pattern

- **Automatic redirect** from one route to another
- Uses **<Navigate />** component
- **/** redirects to **/dashboard**

```
1 import { Navigate } from "react-router";  
2  
3 <Routes>  
4   <Route path="/" element={<Navigate to="/dashboard" replace />} />  
5   <Route path="/dashboard" element={<Dashboard />} />  
6 </Routes>;
```


Protected Routes

- Some routes should **only be accessible** when the user is **authenticated**
- If not authenticated → redirect to **/login**

```
1 import { Navigate, Outlet } from "react-router";
2
3 const ProtectedRoutes = () => {
4   // Your authorization logic goes here,
5   // maybe load token from localStorage, etc.
6   const isAuthenticated = undefined;
7
8   // wait while your logic checks if the user is authorized
9   if (isAuthenticated === undefined) {
10     return <h5>Loading ...</h5>;
11   }
12
13   if (!isAuthenticated) {
14     // user is NOT authorized, navigate to login
15     return <Navigate to="/login" replace />;
16   }
17
18   // user is authorized, show content
19   return <Outlet />;
20 };
21
22 export default ProtectedRoutes;
```

Check authorization

Wait for authorization check

User is NOT authorized

User is authorized

```
1 import ProtectedRoutes from "../ProtectedRoutes";
2
3 <Routes>
4   <Route path="/login" element={<Login />} />
5   <Route element={<ProtectedRoutes />}>
6     <Route path="/dashboard" element={<Dashboard />} />
7   </Route>
8 </Routes>;
```

Data Fetching

Why Data Fetching is hard



What is “server state”?

- Server state is data that lives outside of your website
- It is owned and controlled by a server / API
- Your website only fetches a copy of it

React needs to:

- Load the data
- Handle errors
- Re-fetch data when it goes stale (outdated)



client-state vs. server-state

	Client-state	Server-state
Lives	in the React App	outside the React App
Controlled by	the UI	an API
Can be changed by	user interactions	anything at any time
Stored	with useState	not, requires fetching & synching



Access server-state with `useEffect` + `fetch`

- **Classic approach:** fetch data after a component gets first rendered
- You need to manage:
 - Loading state
 - Error state
 - Storing data

Common Problems with useEffect + fetch

- Copy boilerplate code
- No caching
- No automatic re-fetch

```
1 import { useEffect, useState } from "react";
2
3 const UsersPage = () => {
4   const [users, setUsers] = useState([]);
5   const [loading, setLoading] = useState(true);
6   const [error, setError] = useState(null);
7
8   useEffect(() => {
9     setLoading(true);
10    setError(null);
11
12    fetch("/api/users")
13      .then((res) => res.json())
14      .then((res) => {
15        setUsers(res);
16        setLoading(false);
17      })
18      .catch((err) => setError(err));
19  }, []);
20
21  if (loading) {
22    return <div>Loading...</div>;
23  }
24
25  if (error) {
26    return <div>Something went wrong.</div>;
27  }
28
29  return (
30    <ul>
31      {users.map((u) => (
32        <li key={u.id}>{u.name}</li>
33      ))}
34    </ul>
35  );
36 };
37
38 export default UsersPage;
```

Manage states manually

Handle request manually

Catch exceptions manually

Loading & Error States

```
1 import { useEffect, useState } from "react";
2
3 const UsersPage = () => {
4   const [users, setUsers] = useState([]);
5   const [loading, setLoading] = useState(true);
6   const [error, setError] = useState(null);
7
8   useEffect(() => {
9     setLoading(true);
10    setError(null);
11
12    fetch("/api/users")
13      .then((res) => res.json())
14      .then((res) => {
15        setUsers(res);
16        setLoading(false);
17      })
18      .catch((err) => setError(err));
19  }, []);
20
21  if (loading) {
22    return <div>Loading...</div>;
23  }
24
25  if (error) {
26    return <div>Something went wrong.</div>;
27  }
28
29  return (
30    <ul>
31      {users.map((u) => (
32        <li key={u.id}>{u.name}</li>
33      ))}
34    </ul>
35  );
36 };
37
38 export default UsersPage;
```

This will be replaced by a library

Show user while data is loading

Show user that an exception happened

Show fetched data

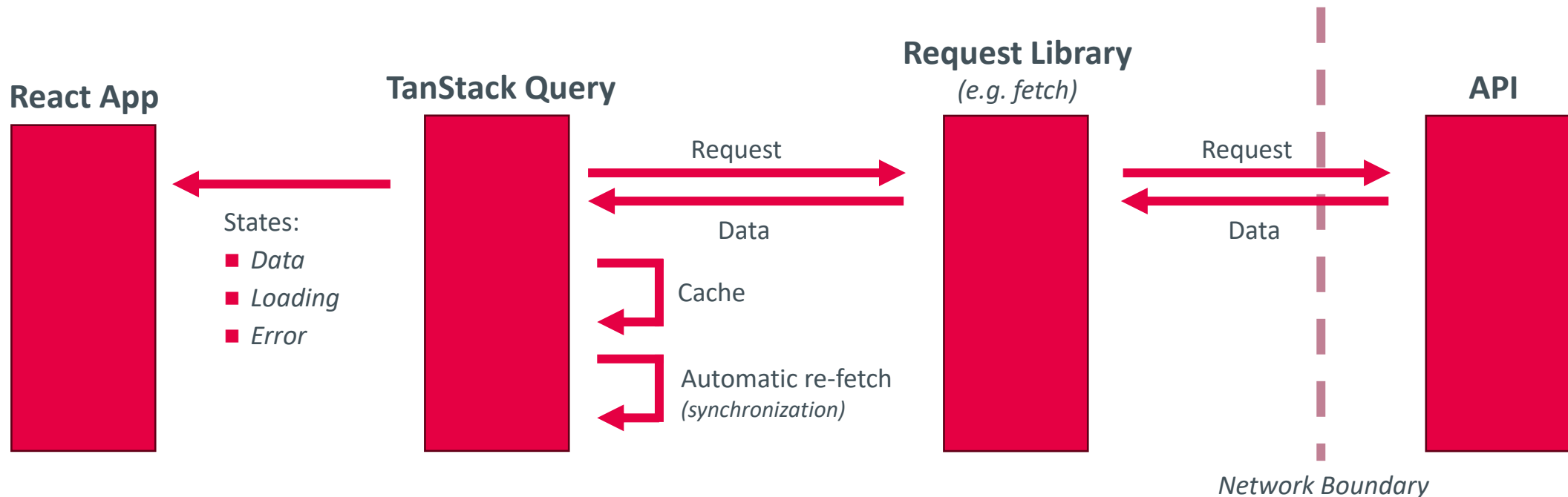
What is TanStack Query?

What is TanStack Query?

- **TanStack Query** is a library for managing **server state**
- It handles **data fetching, caching** and **synchronization**
- Built specifically for **APIs** (*e.g. REST*)
- Works in **addition** to **request libraries** like *fetch* or *axios*

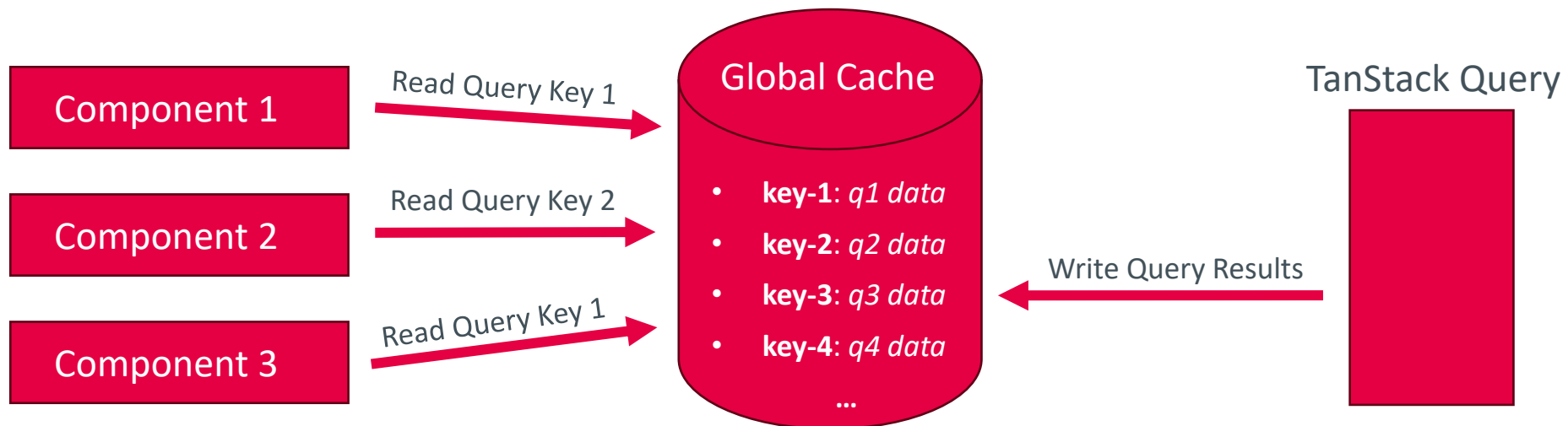
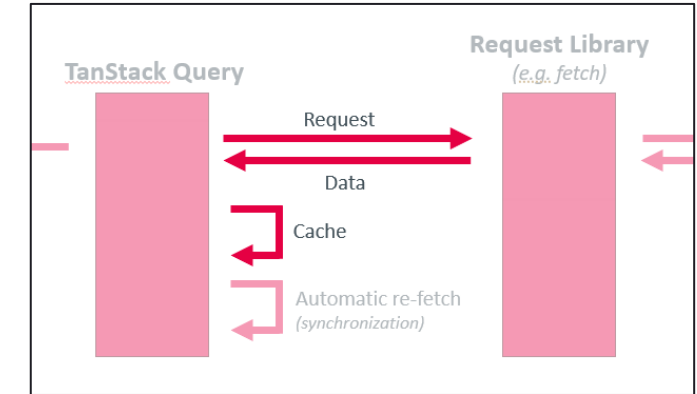
Execute in your project root:

```
npm i @tanstack/react-query
```



Core Idea: Queries & Cache

- A **query** represents **one piece** of **server data**
- Each query is identified by a **query key**
- Query results are stored in a **global cache**
- Components read data **from the cache**, not directly from the server





Mental Model

- TanStack Query **manages all server-data** for you
- **You** still need to **implement** the **network requests** (*with e.g. fetch*)
- **One piece of server data** is represented by a **query**
- **Query results** are stored in a **cache** and can be accessed by a **query key**



When to use TanStack Query

- When you need to **access data** from an **API** (*request/response*)
 - *e.g. REST*
- **Shared data**
 - *across multiple components*
- **Background updates**
 - *automatic re-fetch without manual `useEffect` logic*

When NOT to use it

■ Push-based APIs (*Real-Time*)

- *e.g. Websockets*

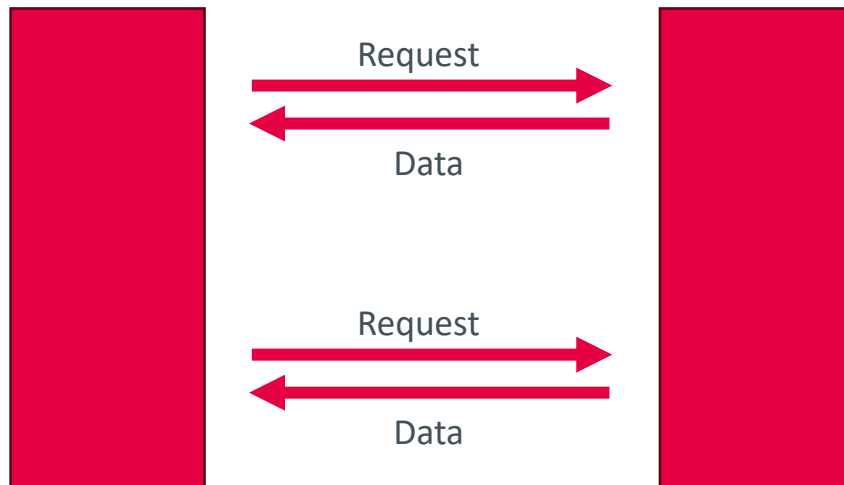


TanStack **cannot** work by default with **persistent connections**

Request/Response APIs

Request Library

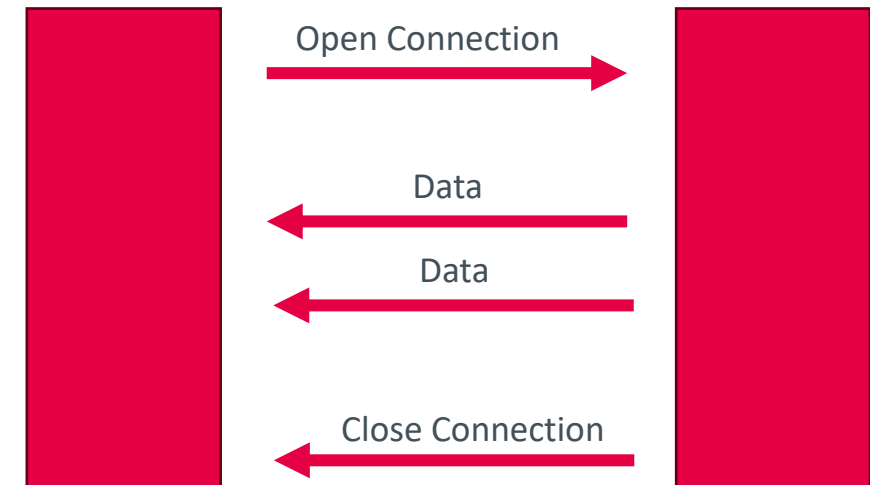
API



Push-based APIs

Request Library

API



Query Basics

QueryClient & QueryClientProvider

- The **QueryClient** manages the *cache, background re-fetching, query lifecycle*
- The **QueryClientProvider** makes it **available** to the app

```
1 import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
2
3 function App() {
4   const queryClient = new QueryClient();
5
6   return (
7     <QueryClientProvider client={queryClient}>
8       {/* YOUR CODE */}
9     </QueryClientProvider>
10  );
11 }
12
13 export default App;
```


useQuery Hook

- **useQuery** fetches data, caches it and keeps it in sync
- It also handles **loading** (*pending*), **error** and **success** automatically
- **useQuery(object) → object**

```
1 const { data } = useQuery({
2   queryKey: ["users"],
3   queryFn: async () => {
4     const res = await fetch("example.com/api/users");
5     if (!res.ok) {
6       throw new Error("request failed");
7     }
8
9     return res.json();
10  },
11 });
```

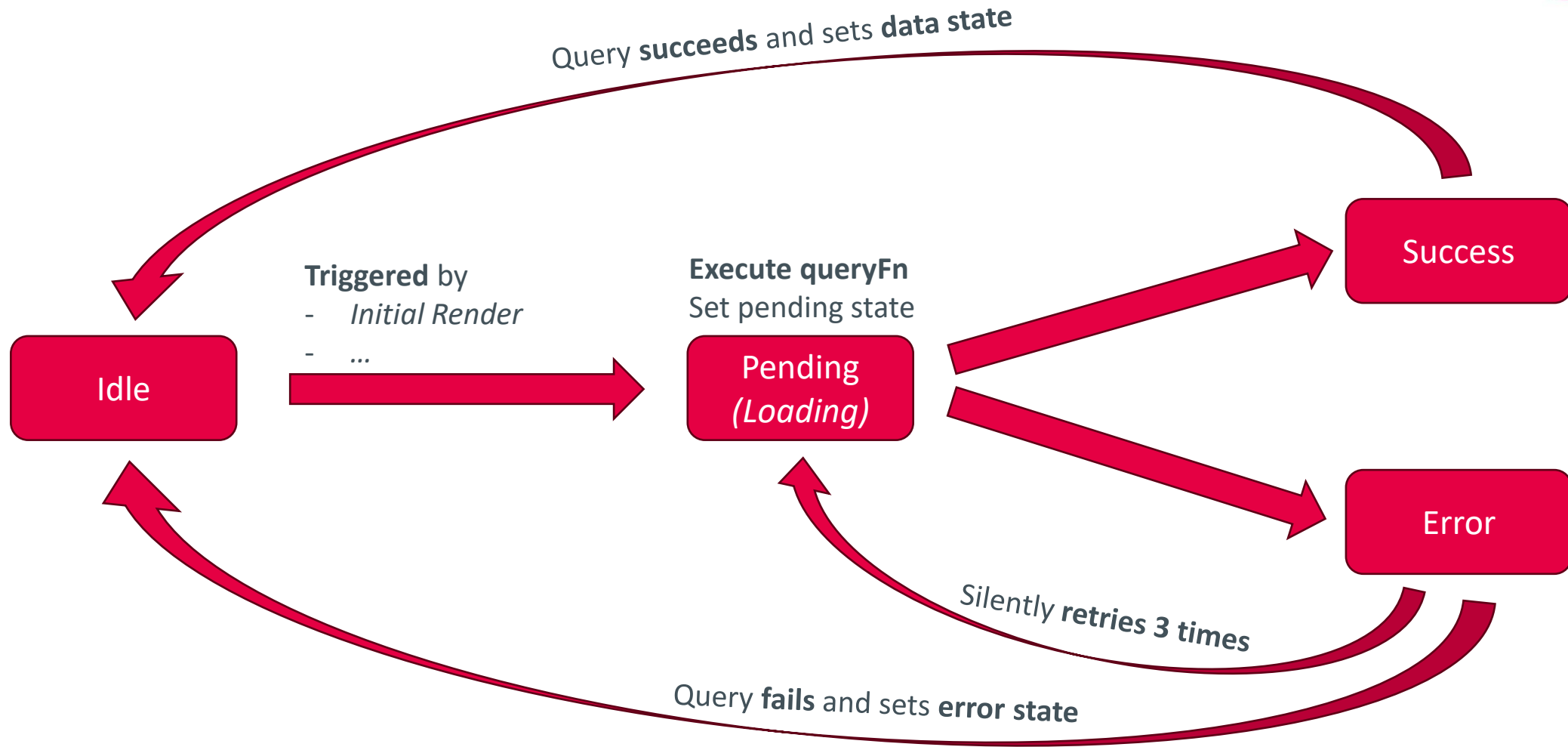
Query Keys

- Query keys identify the data in the cache
- Keys are arrays, not strings



If **two different** queries have the **same key**, the **second query** will **not execute** its query function.

Query Lifecycle



Loading & Error Handling

```
1 import { useQuery } from "@tanstack/react-query";
2
3 const UsersPage = () => {
4   const { data, isPending, error } = useQuery({
5     queryKey: ["users"],
6     queryFn: async () => {
7       const res = await fetch("/api/users");
8       if (!res.ok) {
9         throw new Error("request failed");
10      }
11
12      return res.json();
13    },
14  });
15
16  if (isPending) {
17    return <div>Loading...</div>;
18  }
19
20  if (error) {
21    return <div>Error loading users</div>;
22  }
23
24  return (
25    <ul>
26      {data.map((u) => (
27        <li key={u.id}>{u.name}</li>
28      ))}
29    </ul>
30  );
31 };
```

Define query

Show user a loading screen

Show user an error screen

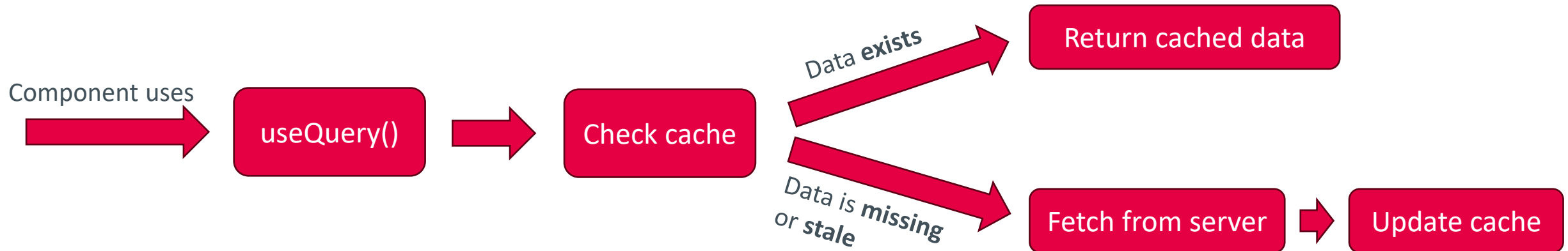
Display the data

Caching & Re-fetching

Cache Behavior

- Query results are stored in a global cache **in memory** (*per browser tab*)
- **Different components** can access the **same cache** (*i.e. access the same data*)

What happens when a query runs





Stale vs. Fresh Data

- Cached data can be **fresh** or **stale**
- “*Stale*” does not mean “*invalid*”
- It only means: *data may be outdated*

Fresh data

- Recently fetched
- Within the **stale time**
 - *Amount of time until data is considered stale*
- **No** automatic re-fetch needed

Stale data

- **Older** than **stale time**
- Still shown to the user
- **May** trigger background **re-fetch**



Re-fetching Strategies

- TanStack Query **re-fetches** data **automatically** to keep it up-to-date
- **Re-fetching** happens based on **defined triggers**
- You **usually** don't need to fetch manually again

Re-fetch triggers *(only applies to stale data)*

- On **mount** *(first render of a component)*
- On **window focus** *(user switches back to the tab)*
- On **reconnect** *(network connection is restored)*
- **Query invalidation** *(e.g. after a mutation)*

Manual Re-fetching

```
1 import { useQuery } from "@tanstack/react-query";
2
3 const Users = () => {
4   const { data, isPending, error, refetch } = useQuery({
5     queryKey: ["users"],
6     queryFn: async () => {
7       const res = await fetch("/api/users");
8       if (!res.ok) {
9         throw new Error("request failed");
10      }
11
12      return res.json();
13    },
14  });
15
16  if (isPending) {
17    return <div>Loading...</div>;
18  }
19
20  if (error) {
21    return <div>Error loading users</div>;
22  }
23
24  return (
25    <>
26      <button onClick={() => refetch()}>Refresh</button>
27
28      <ul>
29        {data.map((u) => (
30          <li key={u.id}>{u.name}</li>
31        ))}
32      </ul>
33    </>
34  );
35 };
```

1. Re-fetch method

Define query

Loading & Error

2. Something to trigger manual re-fetching

Display the data

Mutations



What are Mutations?

- **Mutations** are used to **change server data**
- They represent **write operations**, not reads
 - *e.g. creating a new user, update a profile, deleting an item, etc.*

How mutations differ from queries

- **Queries** → read & cache data
- **Mutations** → send changes to the server
- Mutations do **not** run automatically
- They are triggered by **user actions**

useMutation Hook

```
1 const { mutate } = useMutation({
2   mutationFn: async (newUser) => {
3     const res = await fetch("/api/users", {
4       method: "POST",
5       headers: { "Content-Type": "application/json" },
6       body: JSON.stringify(newUser),
7     });
8
9     if (!res.ok) {
10       throw new Error("request failed");
11     }
12
13     return res.json();
14   },
15 });
```

Updating Server Data

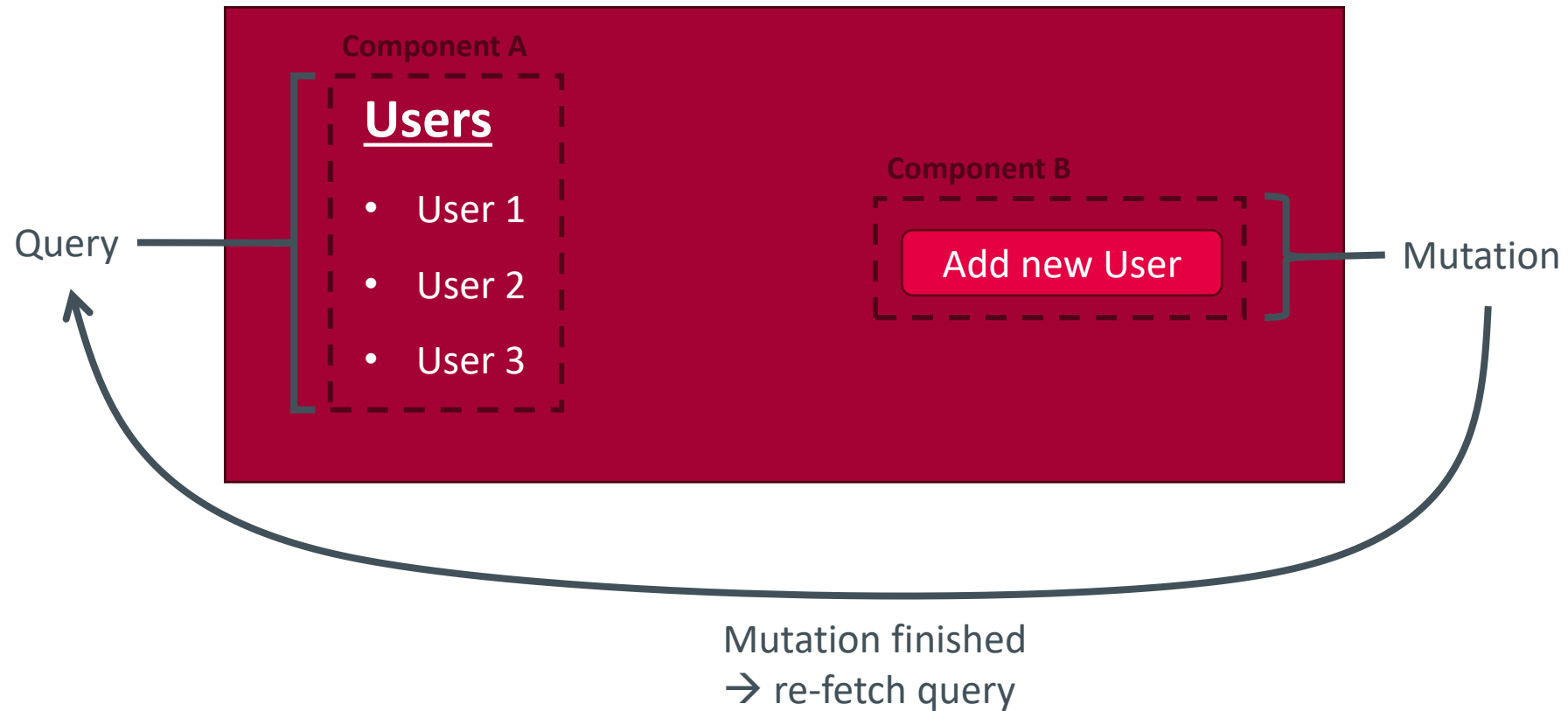
```
1 import { useMutation } from "@tanstack/react-query";
2
3 const CreateUserButton = () => {
4   const { mutate, isPending } = useMutation({
5     mutationFn: async (newUser) => {
6       const res = await fetch("/api/users", {
7         method: "POST",
8         headers: { "Content-Type": "application/json" },
9         body: JSON.stringify(newUser),
10      });
11
12      if (!res.ok) {
13        throw new Error("request failed");
14      }
15
16      return res.json();
17    },
18  });
19
20   return (
21     <button onClick={() => mutate({ name: "Alice" })} disabled={isPending}>
22       Create user
23     </button>
24   );
25 };
```

Define mutation

Call mutate function

Invalidating Queries

■ **Use Case:** *Display list of users + Button to add another user*



Invalidating Queries

- Once the **mutation finished** → the **query key** needs to be **invalidated**
- This causes TanStack Query to **automatically re-fetch** the query
 - *updating the user list with the new user*

Query

```
1 const { data } = useQuery({
2   queryKey: ["users"],
3   queryFn: async () => {
4     const res = await fetch("example.com/api/users");
5     if (!res.ok) {
6       throw new Error("request failed");
7     }
8
9     return res.json();
10  },
11 });
```

Mutation

```
1 import { useMutation, useQueryClient } from "@tanstack/react-query";
2
3 const queryClient = useQueryClient(); ! Get the query client
4
5 const { mutate, isPending } = useMutation({
6   mutationFn: async (newUser) => {
7     const res = await fetch("/api/users", {
8       method: "POST",
9       headers: { "Content-Type": "application/json" },
10      body: JSON.stringify(newUser),
11    });
12    if (!res.ok) {
13      throw new Error("request failed");
14    }
15
16    return res.json();
17  },
18   onSuccess: () => {
19     queryClient.invalidateQueries({
20       queryKey: ["users"],
21     });
22   },
23 });
```

**Once the mutation is successful
→ Invalidate the query**

Summary



Summary

■ Routing with React Router

- *What is Routing?*
- *Basics*
- *Nesting & Layouts*

■ Data Fetching with TanStack Query

- *What is server state?*
- *Queries Basics*
- *Caching + Refetching*
- *Mutations*



TECHNISCHE HOCHSCHULE
OSTWESTFALEN-LIPPE
UNIVERSITY OF
APPLIED SCIENCES
AND ARTS

Thank you for the attention!